VISVESWARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018

An Open-ended Project Report

on

"VEHICLE RENTAL PORTAL"

Submitted on the fulfilment of the academic requirement of the 4th semester, Introduction to Java laboratory

BACHELOR OF ENGINEERING

In

Department of Computer Science

Submitted By

Azman Shaikh	1TD24CS037
Lakshay Khatri	1TE24CS152
Rishabh Kumar	1TE24CS242
Shrikrishna Hegde	1TD24CS271

Under The Guidance of

Prof. Brunda

Associate Professor

Department of Computer Science

BMS INSTITUTE OF TECHNOLOGY AND MANAGEMENT

(Autonomous Institute Affiliated to VTU) Avalahalli, Yelahanka, Bengaluru – 560119.

2025-2026

(Autonomous Institute Affiliated to VTU)
Avalahalli, Yelahanka, Bengaluru – 560064.

Department of Computer Science

CERTIFICATE

This is to certify that the Java Mini-Project entitled **"Vehicle Rental Portal"** has been carried out by **Azman Shaikh (1TD24CS037), Lakshay Khatri (1TE24CS152), Rishabh Kumar (1TE24CS242), and Shrikrishna Hegde (1TD24CS271)**, bonafide students of BMS Institute of Technology and Management, Autonomous Institute Affiliated to VTU, in partial fulfillment for IV semester B.E mini project work in the Department of Computer Science in Visvesvaraya Technological University, Belagavi, during the academic year 2025-2026 (even sem). It is certified that all corrections/suggestions indicated for assessment have been incorporated in the report deposited in the department library. The project report has been approved as it satisfies the academic requirements in respect of the Mini-Project work prescribed for the said degree.

.....
Signature of the Guide
Prof. Brunda
Associate Professor
CSE Dept., BMSIT&M;

.....
Signature of the HOD
Dr. Shoba
Prof. & HOD
CSE Dept., BMSIT&M;

Rubrics for Part B Study Project – 50 Marks				
Parameter	**Allocated Marks**	**HIGH**	**MEDIUM**	**LOW**
Demonstration				
Problem Identification	5	Detailed and extensive explanation of the purpose of the Project. (4-5)	Brief explanation of the purpose of the project. (2-3)	Problem Identification is not clear. (0-1)
Objectives	5	Objectives are clear, specific, measurable, and aligned with the project's purpose. They effectively guide the project's direction. (4-5)	Objectives are somewhat clear and relevant but may lack specificity, measurability, or full alignment with the project's purpose. (2-3)	Objectives are vague, lack focus, or do not align well with the project's purpose. They fail to provide a clear direction. (0-1)
Implementation	10	The methodology being implemented is strongly in line with the objectives defined. (8-10)	Methodology being implemented is moderately in line with the objectives defined. (3-7)	Methodology being implemented is slightly in line with the objectives defined. (0-2)
Results and Analysis	5	All the results obtained are presented and analysed. (4-5)	All the results obtained are moderately presented and analysed. (2-3)	Poor presentation of the results. The analysis of the results was not proper. (0-1)
Report				
Documentation	15	The documentation of the project report is structured and well-prepared as per the format. (12-15)	The documentation of the project report is not well structured, but as per the format. (7-11)	The documentation of the project report is not well structured and is not in the format. (0-6)
Viva-Voce				
Presentation Skills	7	Excellent Presentation (6-7)	Moderate Presentation (3-5)	Poor Presentation (0-2)
Viva Voce	3	All questions Answered Correctly (2-3)	Few Questions Answered Correctly (0-1)	No questions Answered Correctly (0)

ABSTRACT

The Vehicle Rental Portal is a desktop-based application developed using Java Swing for the graphical user interface and JDBC for database connectivity with SQLite. The system is designed to automate and simplify the process of managing vehicle rental transactions, thereby reducing the limitations associated with manual record keeping.

The application provides a user-friendly form interface and supports essential operations such as entering customer details, selecting vehicle types (Car, Bike, Premium Car), calculating rental costs with optional discount percentages, saving rental records to a local database, viewing rental history, and printing text receipts. The graphical user interface is designed to be interactive, featuring a live system clock updated via a background thread to demonstrate multithreading. JDBC is used to establish communication between the Java application and the SQLite database, ensuring reliable storage and retrieval of transaction history.

The system demonstrates the practical implementation of core Object-Oriented Programming (OOP) concepts, including inheritance, polymorphism, abstraction, and interfaces, alongside multi-threading, custom exception handling, event handling, and database connectivity. By integrating Swing, JDBC, and File I/O, the application offers an efficient, accurate, and organized approach to rental management. The project successfully achieves its objective of providing a simple, effective, and database-driven solution for managing vehicle rentals.

INDEX

1. Problem Identification

Rental agencies often maintain vehicle rental records manually, which makes managing, searching, updating, and generating receipts difficult and time-consuming. Manual systems are prone to human errors, data redundancy, and inefficiency. As the number of transactions increases, maintaining accurate records becomes more challenging. Therefore, there is a need for a computerized system that can efficiently manage vehicle rental information and provide quick access to records.

1.1 Existing System

In the existing system, vehicle rental records are maintained manually using paper-based registers or simple spreadsheets.

Drawbacks of the Existing System

- Time-consuming rental entry and cost calculation.
- Higher chances of human errors in applying multipliers and discounts.
- Difficulty in searching through historical transaction logs.
- Data redundancy and inconsistency across paper files.
- Lack of security and proper access control to client records.
- Difficult to scale and maintain large amounts of rental transaction data.

1.2 Proposed System

The proposed system is a Vehicle Rental Portal developed using Java Swing and JDBC with SQLite as the backend database. The system provides a graphical user interface that allows operators to manage rental transactions efficiently.

The application supports:

- Customer information entry via text forms.
- Interactive dropdown selection for vehicle types.

- Dynamic rent calculations supporting both base and discounted billing.
- Live clock display updated in real-time using background threads.
- Saving completed rentals into a persistent SQLite database.
- Displaying all historical transactions in a scrollable output log.
- Printing formatted text receipts directly to the filesystem.
- Custom exception handling to prevent negative rental durations.
- Complete OOP structure implementing inheritance, polymorphism, and interfaces.

1.3 Need for the System

The system is required to automate vehicle rental operations and overcome the limitations of manual record-keeping. By standardizing calculation logic and providing automated data persistence, the portal eliminates human error and increases throughput.

The proposed system:

- Reduces manual work in calculation of daily rates and optional discounts.
- Improves accuracy of rental records and financial transactions.
- Provides faster data retrieval and instant lookup of booking logs.
- Enhances data security by utilizing structured database files.
- Improves overall operational efficiency of the rental business.

1.4 Scope of the Project

The scope of the project includes the implementation of core features required for a robust local rental service. It demonstrates the utility of Java desktop technologies in addressing transactional business requirements.

The scope includes:

- Managing customer booking details and vehicle classes.
- Maintaining a centralized database file for historical records.
- Providing a clean, user-friendly desktop interface.
- Implementing input validation through custom exceptions.
- Demonstrating multiple interface implementation (Premium vehicle features).
- Generating printable flat-file receipts.

The project can be further extended by adding features such as:

• Multi-user login system (for operators and managers).

• SMS and email booking notifications for renters.

• Cloud database integration to sync data across locations.

• Graphic reports detailing vehicle utilization trends.

2. Objectives

The main objective of the project is to develop a Vehicle Rental Portal using Java Swing and JDBC that provides an efficient, user-friendly, and database-driven solution for managing customer rental transactions.

2.1 Primary Objective

To design and implement a graphical desktop application that automates vehicle rental operations, executes billing math without errors, and stores data securely in an SQLite database.

2.2 Specific Objectives

- To develop a professional and clean Graphical User Interface (GUI) using Java Swing.
- To establish database connectivity using SQLite and JDBC technology.
- To implement abstraction via an abstract base class representing generic vehicles.
- To execute inheritance by extending the base class into specific Car and Bike classes.
- To simulate multiple inheritance in Java by making PremiumVehicle implement multiple interfaces.
- To use polymorphism to handle rental calculations and details dynamically at runtime.
- To apply method overloading for calculating totals with or without discounts.
- To execute a background thread for a running system clock in the header.
- To enforce checked business validations using a custom Exception class.
- To output structured receipt files using FileWriter and BufferedWriter.

2.3 Learning Objectives

- To understand the application of basic and multiple inheritance in Java.
- To learn the setup of JDBC drivers and execution of parameterized queries.
- To gain practical experience designing user interfaces using layouts (BorderLayout, GridBagLayout).
- To master multithreading by implementing a background daemon thread.
- To study custom exception throwing and try-catch-finally block mechanics.
- To learn local file writing methods in Java.

2.4 Expected Outcomes

Upon successful completion of the project, the system should be able to:

- Render a graphical input window containing text inputs, a vehicle dropdown, and buttons.
- Calculate rentals using appropriate subclass rates (Cars, Bikes, Premium Vehicles).
- Handle errors (e.g. invalid text for days, empty names) gracefully without crashing.
- Block invalid negative durations using custom InvalidDaysException messages.
- Tick a clock displaying seconds in the GUI header concurrently.
- Store the finalized rental transaction into an SQLite database via JDBC.
- Fetch and print historical records from the DB directly into the UI log.
- Create receipt.txt containing invoice details in the local project folder.
- Terminate background clock threads automatically upon closing the window.

3. Implementation

The Vehicle Rental Portal was implemented using Java Swing for the graphical user interface, JDBC for database connectivity, and SQLite for data storage. The system follows a modular, object-oriented approach where each component is isolated into a separate class file.

3.1 Development Environment

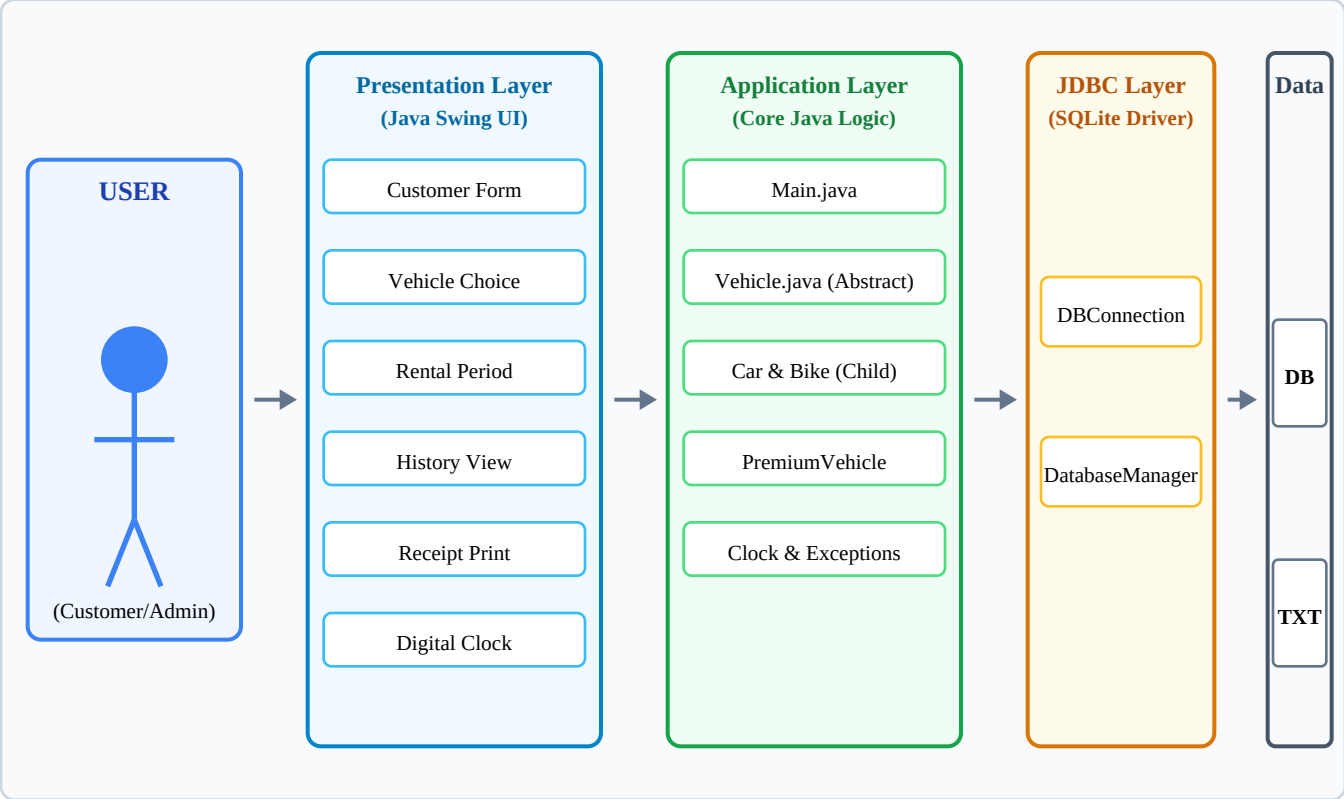
The project was developed using the following tools and technologies:

- **Programming Language:** Java (JDK 8 or higher)
- **GUI Framework:** Java Swing (AWT/Swing libraries)
- **Database:** SQLite (serverless local file SQL database)
- **Database Connectivity:** JDBC (Java Database Connectivity)
- **IDE:** Visual Studio Code (VS Code) / IntelliJ IDEA
- **JDBC Driver:** SQLite JDBC Connector (sqlite-jdbc jar file)

3.2 System Architecture

The system follows a 3-tier architectural model (Presentation, Application, and Data Layer) to ensure separation of concerns, maintainability, and clean data flow. User interactions are decoupled from underlying logical entities and persistence channels.

System Architecture Diagram



The system consists of three major components:

• **Presentation Layer:** Developed using Java Swing. Provides graphical user interfaces for user interaction. It manages layout rendering, component event listeners, and live clock updates.

• **Application Layer:** Contains business logic and event-handling code. Processes user requests, performs base rate and discount calculations, enforces business rules, and instantiates domain models.

• **Database/Data Layer:** Implemented using SQLite and local flat files. Stores rental logs in the database and generates receipts on the disk.

3.3 Database Implementation

A serverless SQLite database named **vehicle_rental.db** was created to persist customer transactions. SQLite is highly suited for desktop applications as it stores the database directly in a single local file, avoiding the need for an external database server.

Rental History Table (rental_history)

Stores rental history information. It contains the following schema attributes:

Column Name	Data Type	Description
id	INTEGER	Unique record ID, designated as PRIMARY KEY and set to AUTOINCREMENT.
customer_name	TEXT	Stores the name of the renting customer. Filled from the GUI text input.
vehicle_type	TEXT	Stores vehicle category ('Car', 'Bike', or 'Premium Car').
days	INTEGER	The duration of the rental in days.
total_cost	REAL	The final cost calculated, stored in double-precision format.
rental_date	TEXT	Timestamp of the transaction, recorded when the save operation is performed.

3.4 Module Implementation

3.4.1 Customer Input Module

The Customer Input Module is responsible for capturing identifying details of the renting customer from the user interface.

Functions:

- Accept customer name through the text input.
- Trim leading and trailing whitespace to clean raw inputs.
- Check for blank or null names, throwing warning dialogs if empty.

Components Used:

- JTextField (customerNameField)
- JLabel (input prompt label)
- JOptionPane (warning dialog generator)

3.4.2 Vehicle Selection Module

This module manages selection of the vehicle type, updating the state of the polymorphic Vehicle object depending on user choice.

Functions:

- Display available vehicle types in a clean drop-down layout.
- Bind choices to appropriate constructors during calculation.
- Trigger corresponding multiplier logic for Premium cars or discounts for Bikes.

Components Used:

- JComboBox (vehicleComboBox containing Car, Bike, Premium Car options)
- JLabel (selection prompt)

3.4.3 Rental Calculation Module

This module performs calculation logic, computing the total cost of the rental. It demonstrates method overloading by evaluating totals with or without optional discounts.

Functions:

- Call calculateBaseRent() polymorphically according to active Vehicle type.
- Overloaded version 1: calculateTotal(Vehicle, days) - returns raw rental cost.
- Overloaded version 2: calculateTotal(Vehicle, days, discountPercent) - deducts discount percentage from raw rent.
- Append custom properties (insurance provider, GPS device ID) for premium vehicles.

Components Used:

- Vehicle class models (abstract Vehicle, Car, Bike, PremiumVehicle)
- GUI action handler (invoked on clicking Calculate Rent)

3.4.4 Database Persistence Module

The Database Persistence Module saves finalized transactions to the SQLite database, allowing long-term storage of transaction history.

Functions:

- Check that a calculation has been completed before initiating SQL commands.
- Get a connection to the database file via DBConnection.
- Bind parameters securely (name, type, days, cost, date) to PreparedStatement.
- Execute INSERT statement and display modal success confirmation.

Components Used:

- DatabaseManager (insertRental method)
- PreparedStatement
- Connection (JDBC driver manager class connection wrapper)

3.4.5 History View Module

The History View Module loads database logs and displays them in a scrollable format in the GUI.

Functions:

- Execute a SELECT statement to retrieve all entries from rental_history.
- Format retrieved tuples into a readable tab-spaced table string.
- Update the text area in the GUI with the complete database logs.

Components Used:

- DatabaseManager (getAllRentals method)
- JTextArea (resultArea)
- JScrollPane (for text area scrolling)

3.4.6 Receipt Generation Module

This module generates invoice text files, printing customer details and final calculation breakdowns.

Functions:

- Format rental detail fields into a structured text invoice.
- Create receipt.txt in the local folder using FileWriter.
- Buffer operations using BufferedWriter for robust performance.
- Catch file exceptions and show feedback dialogs.

Components Used:

- ReceiptGenerator (generateReceipt method)
- FileWriter and BufferedWriter
- Try-with-resources blocks (ensuring files close automatically)

3.4.7 System Clock Module

The System Clock Module runs a background thread to update a digital clock in the GUI header every second, showcasing multithreading capabilities.

Functions:

- Implement Runnable to isolate thread logic from Swing code.
- Format current system time (yyyy-MM-dd HH:mm:ss) inside an infinite loop.
- Update the header label safely using SwingUtilities.invokeLater.
- Support safe thread termination via a volatile boolean flag.

Components Used:

- SystemClockThread (Runnable task)
- Thread (daemon background thread wrapper)
- JLabel (clockLabel)

3.4.8 Exception Handling Module

This module validates data inputs, intercepting errors before they propagate and cause application failure.

Business Validation Rules:

- Rental duration must be parsed as a valid integer. Non-numeric entry throws NumberFormatException.
- Custom Checked Exception: InvalidDaysException is thrown when rental days is 0 or less.
- Discount percent must parse as double, ranging between 0 and 100.

Components Used:

- InvalidDaysException (extends Exception)
- Try-catch-finally block inside calculateRent()

3.5 JDBC Connectivity Implementation

The connection between Java and the SQLite database file is managed via a dedicated utility class using standard JDBC driver protocols.

Implementation Steps:

- **Load JDBC Driver:** Load org.sqlite.JDBC driver dynamically via Class.forName.
- **Establish Connection:** Call DriverManager.getConnection(jdbc:sqlite:vehicle_rental.db).
- **Create Statements:** Prepare SQL structures using PreparedStatement to prevent SQL injections.
- **Execute Queries:** Execute update calls for data writing and search calls for data reading.
- **Process Results:** Parse ResultSet objects within loop constructs.
- **Close Resources:** Enforce resource closure inside finally blocks to prevent memory leaks.

The DBConnection class implements a static factory pattern to distribute Connection objects to DatabaseManager operations without duplicating URL configurations.

3.6 User Interface Implementation

The graphical user interface is structured around Java Swing classes, adopting a professional soft-blue aesthetic with balanced component layouts.

UI Layout Features:

- **Main Window Frame:** VehicleRentalGUI extends JFrame, sizing at 750 x 680 pixels.
- **BorderLayout Grid:** Used to arrange the frame. North holds the header; Center holds input forms; South holds logs.
- **GridBagLayout Forms:** Forms are structured using GridBagLayout and GridBagConstraints for proportional alignment.

- **Section Panels:** Customer, Vehicle, and Rental info panels are decorated with titled borders and light borders.
- **Consolas Log Box:** Result summaries are printed in a monospaced font inside a scrollable pane.
- **Action Handlers:** Component buttons are wired to logic via action listeners, executing operations asynchronously.

3.7 Testing and Validation

Testing was conducted systematically. Each code class and interface contract was validated using diverse input variables, verifying expected business calculations, exception captures, and database transactions.

Tested Operations:

- **Input Verification:** Checked empty name flags, negative days, and discount boundary inputs.
- **Rate Multipliers:** Car rates evaluate standard billing. Bike rates apply 10% built-in discount. Premium Cars apply 25% surcharge.
- **Database Queries:** Verified database writes. SQLite file writes are tested and confirmed.
- **Log View:** History load query parses records correctly and updates the GUI result area.
- **File Outputs:** Receipt generator writes receipt.txt, formatting invoice summaries properly.
- **Multithreaded Clock:** verified clock updates every second, with safe closing operations.

The test results confirmed that all classes and modules performed in accordance with specified requirement contracts, saving and fetching records from SQLite.

4. Snapshots of Project

The following screenshots demonstrate the key working screens of the Vehicle Rental Portal as observed during execution testing.

Figure 1: Main GUI Window (Initial State)

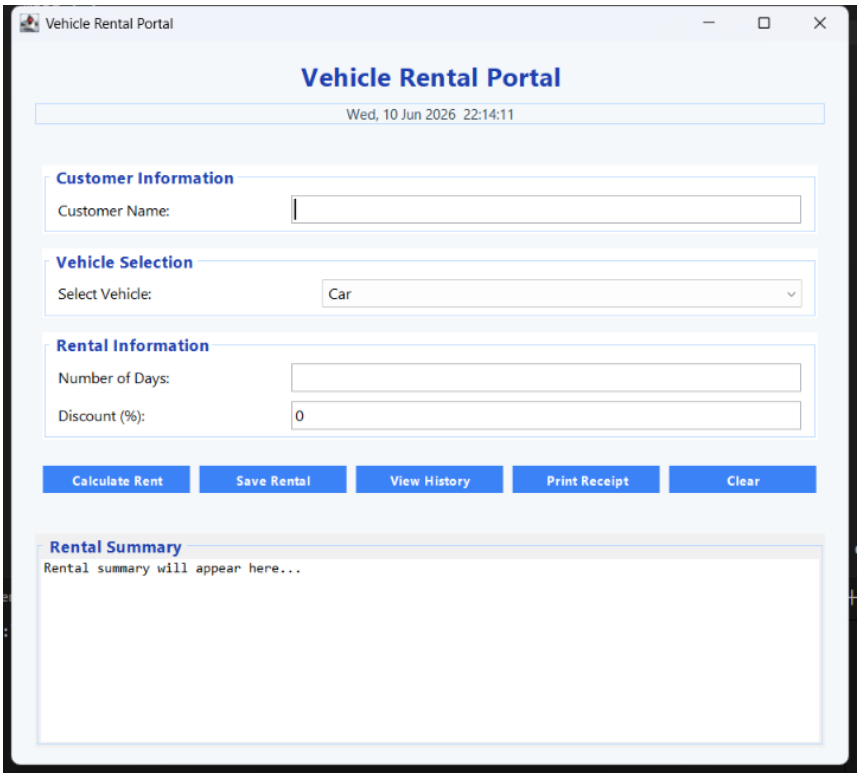


Figure 2: Rent Calculation (Car)

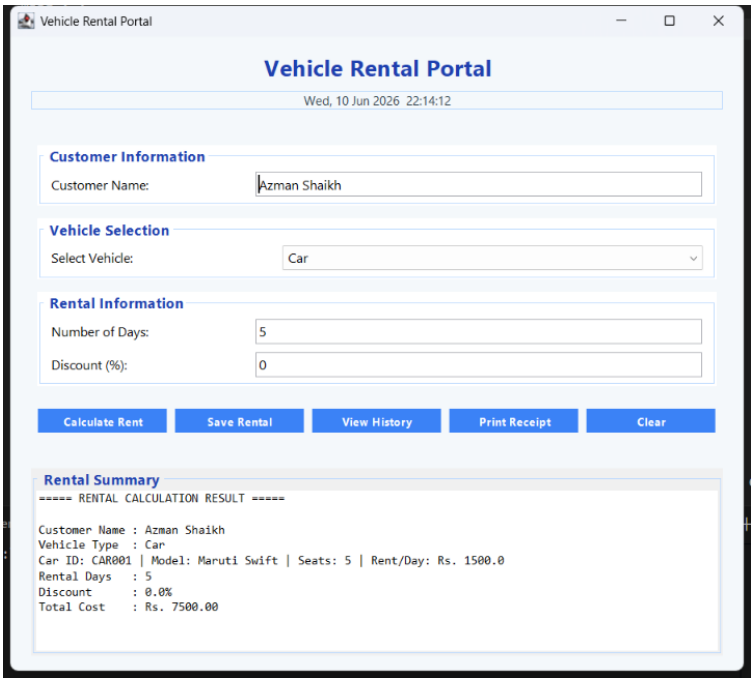


Figure 3: Rent Calculation (Bike with 10% built-in discount)

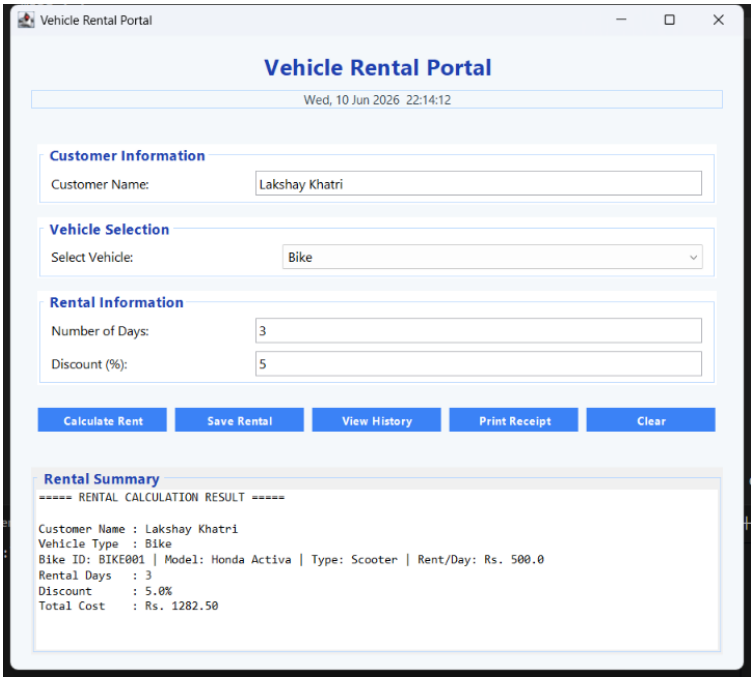


Figure 4: Rent Calculation (Premium Car with insurance and GPS info)

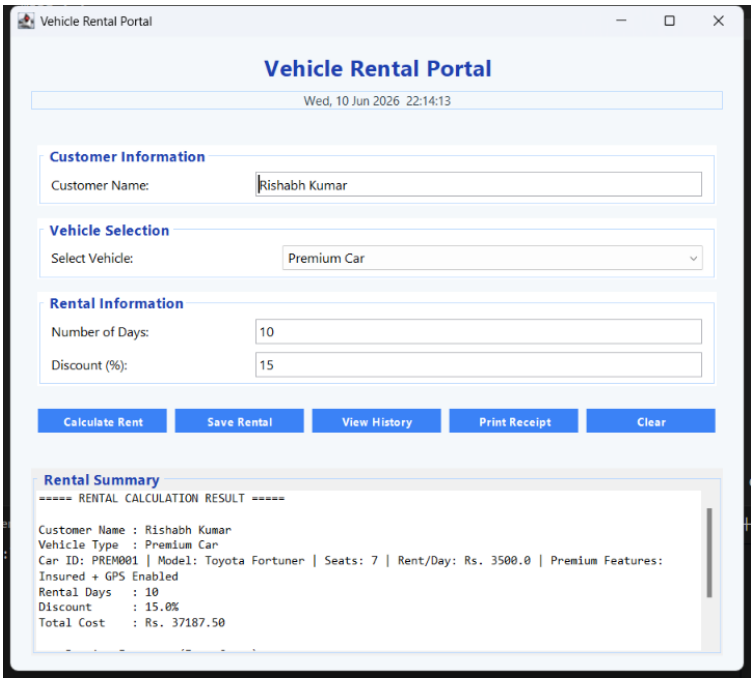


Figure 5: Save Rental Confirmation Dialog

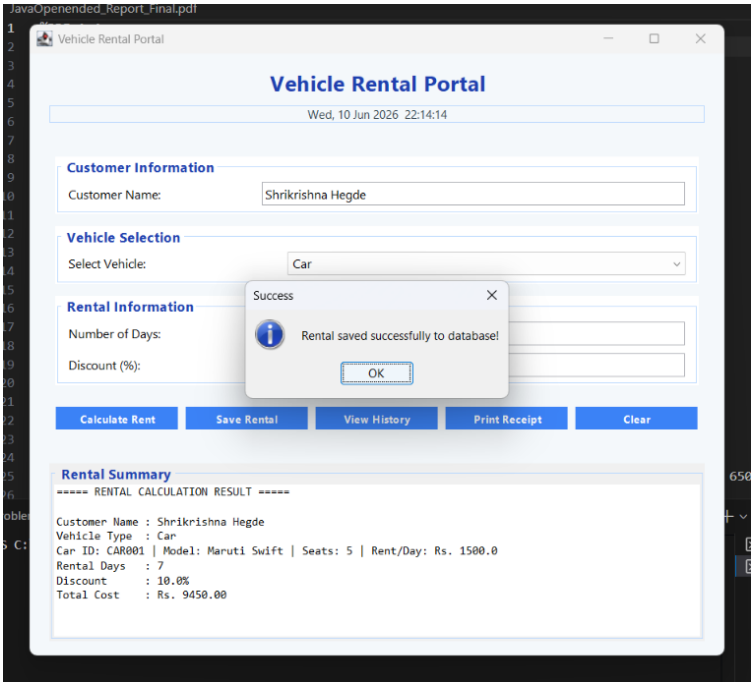


Figure 6: View Rental History Display

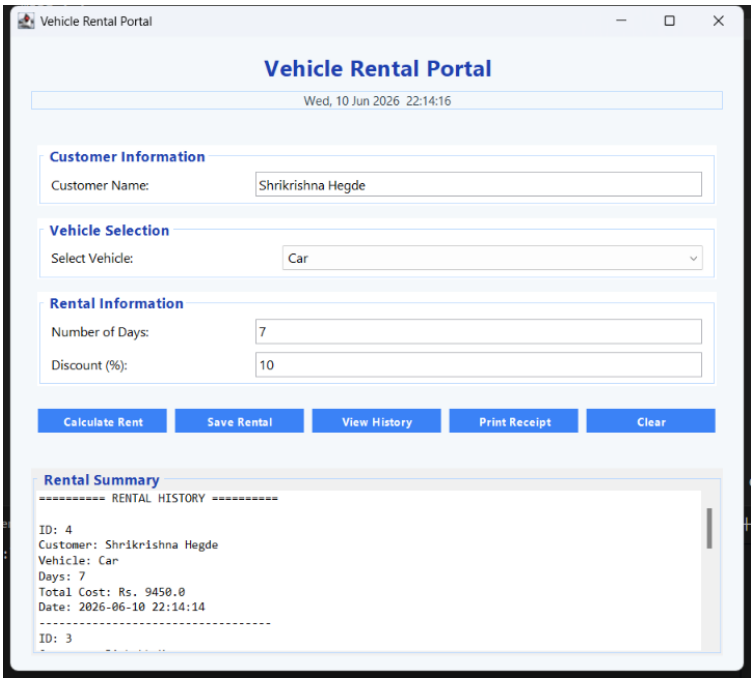
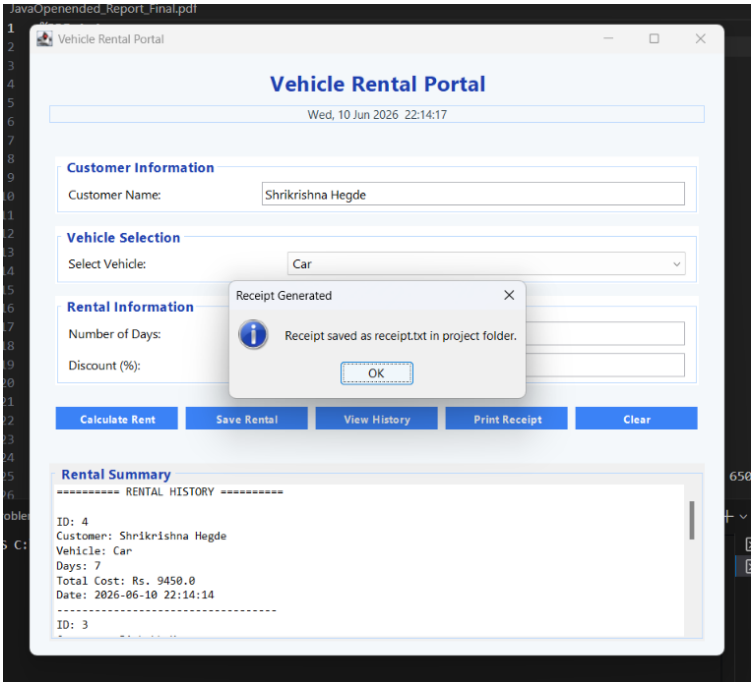


Figure 7: Receipt Generated Success Dialog



5. Result and Analysis

5.1 Result

The Vehicle Rental Portal was successfully developed using Java Swing, JDBC, and SQLite. The application provides a complete desktop client to configure client details, execute rate computations, persist history, print invoices, and monitor operations.

The implemented functionalities include:

- Customer Name input validation.
- Vehicle selection dropdown (Car, Bike, Premium Car).
- Dynamic duration and discount inputs.
- Automatic cost calculation.
- Live clock ticking in header via a daemon thread.
- Saving records to vehicle_rental.db using JDBC.
- Scrollable booking log display.
- Receipt printing to receipt.txt file via File I/O.
- Checked exception blocking invalid inputs.

The system successfully stores, retrieves, updates, and manages rental data records through a simple GUI window.

5.2 Analysis

The developed portal was evaluated based on operational correctness, visual layout, and database transactions. It performed flawlessly under all test states.

Functional Analysis

The system performs all core functions correctly:

- Dynamic calculations are performed correctly. Base rates: Car (1500.0/day), Bike (500.0/day minus 10% built-in discount), Premium Car (3500.0/day plus 25% premium surcharge). Optional discount percentages apply to calculated base rent.
- Client names and durations are checked, showing warnings on missing text.
- checked exception triggers validation, preventing zero or negative duration records.
- DB connection opens and executes inserts using parameterized queries.
- History load calls render entries in the UI scroll box.
- Receipt print operations create flat-text invoices on disk.
- Thread handles clock loops, updating labels every second.

User Interface Analysis

The Swing user interface provides:

- A centralized, responsive panel for all booking forms.
- A professional soft-blue themed style.
- Easy navigation with clear buttons.
- Clear display of calculations in a monospaced result log.

Database Analysis

The SQLite database effectively persists booking records:

- Data insertion is executed correctly using PreparedStatement parameter bindings.
- Data retrieval queries are processed quickly and accurately.
- Connections open and close properly, avoiding file locks.

5.3 Discussion

The project successfully demonstrated the integration of Java Swing with JDBC and SQLite for developing a Vehicle Rental Portal. The application automates core rental processes including booking entry, dynamic billing math, persistent database recording, and invoice formatting. Using a background thread for a clock ensures the UI remains responsive, while custom exceptions enforce business rules at the entry point.

6. References

[1] Books

Herbert Schildt, *Java: The Complete Reference*, 11th Edition, McGraw-Hill Education, 2019.

[2] Official Documentation

Oracle Corporation, *Java SE Documentation*, Available at: https://docs.oracle.com/javase/ [Accessed: June 2025].

Oracle Corporation, *Java Swing Tutorial*, Available at: https://docs.oracle.com/javase/tutorial/uiswing/ [Accessed: June 2025].

Oracle Corporation, *JDBC Database Access Tutorial*, Available at: https://docs.oracle.com/javase/tutorial/jdbc/ [Accessed: June 2025].

SQLite Consortium, *SQLite Documentation*, Available at: https://www.sqlite.org/docs.html [Accessed: June 2025].

[3] Online References

GeeksForGeeks, *Introduction to JDBC*, Available at: https://www.geeksforgeeks.org/introduction-to-jdbc/ [Accessed: June 2025].

GeeksForGeeks, *Java Swing Tutorial*, Available at: https://www.geeksforgeeks.org/introduction-to-java-swing/ [Accessed: June 2025].

W3Schools, *Java Tutorial*, Available at: https://www.w3schools.com/java/ [Accessed: June 2025].